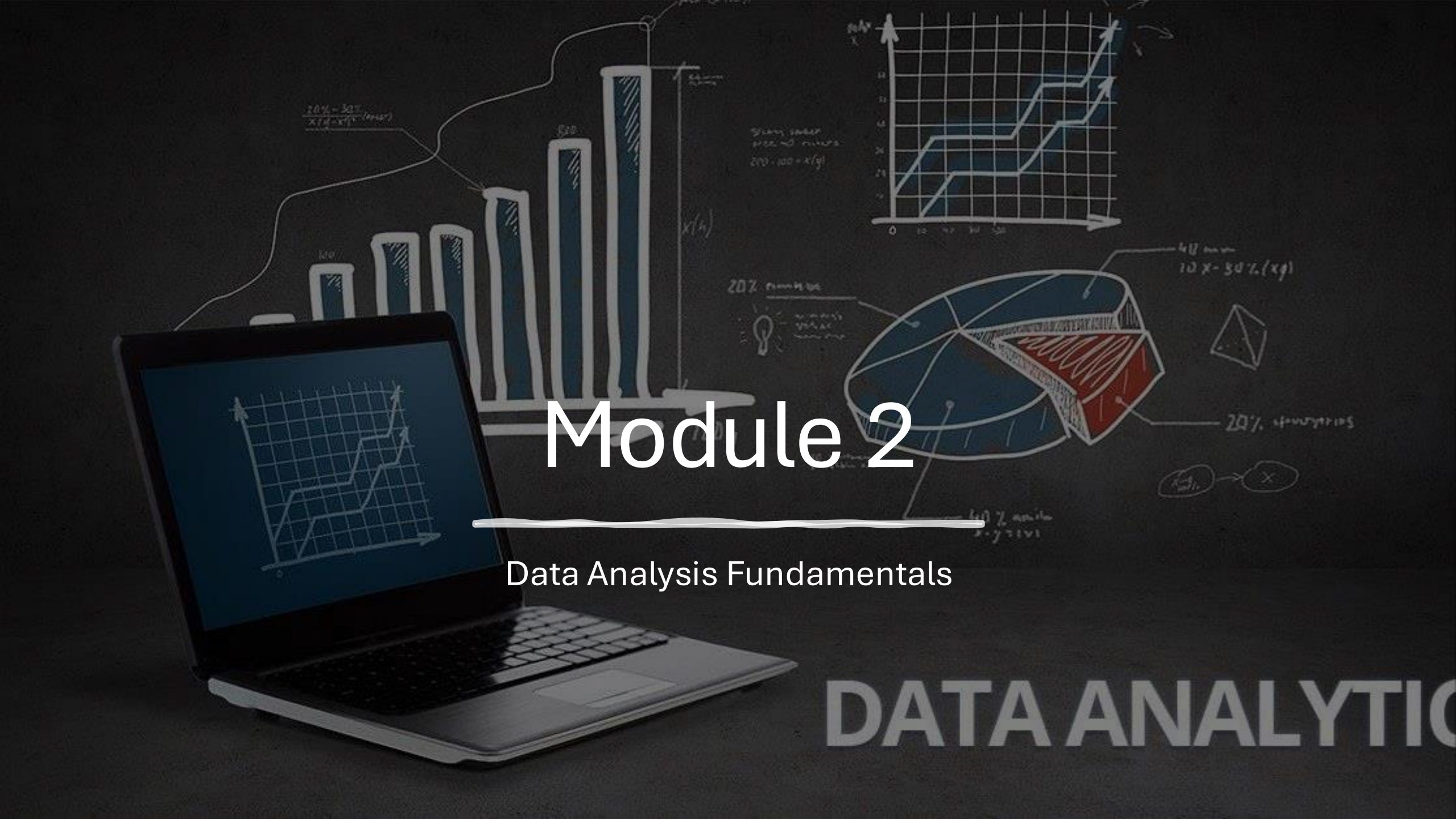




K-Youth Training Program

**Data Analytics  
for business  
growth and  
sustainability in  
the workforce**



# Module 2

Data Analysis Fundamentals

DATA ANALYTICS

**slido**

Please download and install the  
Slido app on all computers you use



**What did you find interesting  
about yesterday's class?**

① Start presenting to display the poll results on this slide.

# Module Contents

## Module 2: Data Analysis Fundamentals

- 1. Databases – types and purpose
- 2. Tools used to ingest data
- 3. Storing and retrieving data
- 4. Data Quality and Cleaning Techniques
- 5. Data Transformation
- 6. Data Integration and ETL Processes
- 7. Python Basics for Non-IT Students
  - a. Introduction to Python and Its Uses in Data Analytics
  - b. Setting Up Python Environment
  - c. Basic Python Syntax and Operations
  - d. Data Structures: Lists, Tuples, Dictionaries
  - e. Importing and Using Libraries (NumPy, Pandas)
  - f. Basic Data Manipulation with Pandas

# Storing and Retrieving Data

Storage is an important step where data can be persisted for further action – transformation, cleaning etc.

Data is usually stored in:

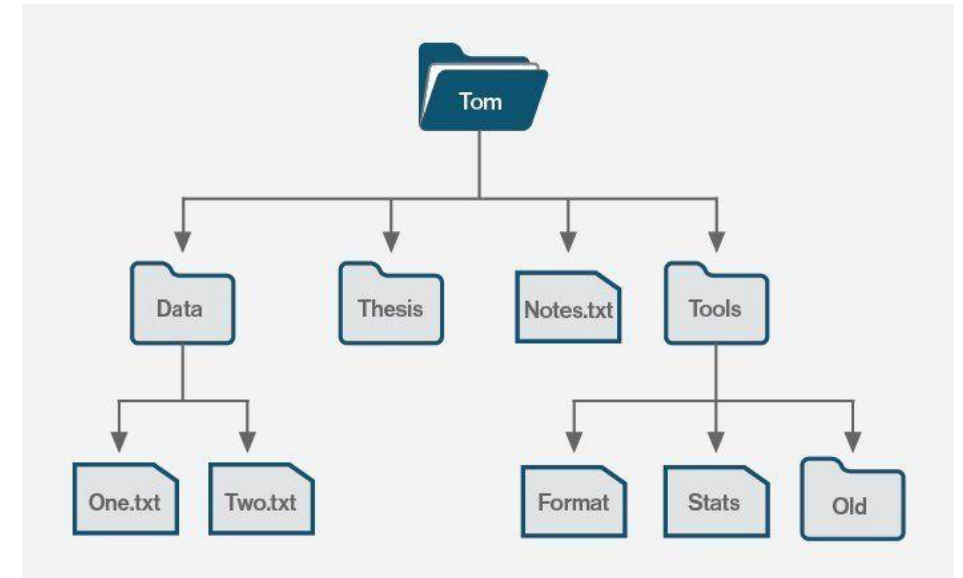
- File System
- Cloud Storage
- Databases

Data can come in many formats as discussed previously

We will look at the various ways data is stored and explore the creation of data and also retrieval of the data

# Data in a file system

- Data in a file system usually is stored in a folder/directory on a hard disk or SSD's
- Data stored can also be organized in a few ways
  - Flat structure – all data resides in a single directory
  - Hierarchical – Data stored in a hierarchical format which contains a root, directories and files.
  - Journalled – changes to the data is maintained to avoid data loss
- Files that are stored also have certain characteristics
  - Stored with an extension .jpg,.txt, .xlsx, .dat etc.
  - Contain metadata like size, permissions, creation/modification data
  - Storage sector – location of the file data on the medium of storage



# Data on Cloud Storage

- Data stored on large distributed servers around the world from providers like Amazon, Google, Azure, Oracle etc.
- Example of data on the cloud include phone contacts backed up to cloud, photos, documents etc.
- Products from Amazon like Amazon S3 is the tool to store files
  - Files are stored in a 'bucket' on the cloud.
  - Within a 'bucket' you can have a folder hierarchy to store files
  - Accessing files require tools or an API to access.
- Create an AWS account <https://aws.amazon.com/>
  - Upon creation we will create a bucket to store a file and find out how to access the data

**General purpose buckets (1)** [Info](#) **All AWS Regions**  [Copy ARN](#) [Empty](#) [Delete](#) [Create bucket](#)

Buckets are containers for data stored in S3.

< 1 > 

|                       | Name ▲                          | AWS Region ▼                         | IAM Access Analyzer                              | Creation date ▼                         |
|-----------------------|---------------------------------|--------------------------------------|--------------------------------------------------|-----------------------------------------|
| <input type="radio"/> | <a href="#">classbd01bucket</a> | Asia Pacific (Sydney) ap-southeast-2 | <a href="#">View analyzer for ap-southeast-2</a> | November 28, 2024, 14:28:59 (UTC+08:00) |

# Data in Databases

- Databases generally have three types
  - SQL / RDBMS
  - NoSQL – Document DB
  - Graph DB
- We will focus on RDMBS and the Graph
- Using Amazon AWS we will create a database and learn some SQL Queries



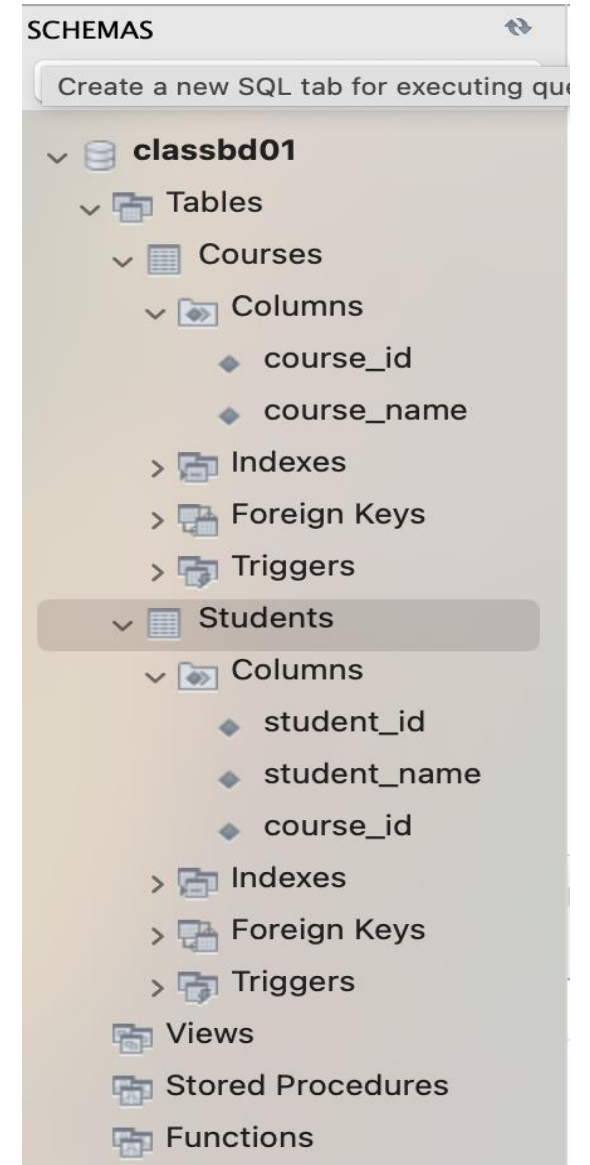
# RDBMS - Basic SQL

- Create a database
  - CREATE SCHEMA <databasename>
  - CREATE DATABASE <databasename>
- Create a table to store data
  - CREATE TABLE students (id INT, name VARCHAR(100), age INT);
- Select data from database
  - SELECT \* FROM <schema>.<tablename>
  - SELECT name, age FROM students;
- Insert data into database
  - INSERT INTO students (name, age) VALUES ('John', 20);
- Update data in a database
  - UPDATE students SET age = 21 WHERE name = 'John';

# RDBMS - Create two or more tables with PK/FK

Example for 2 tables

- **CREATE TABLE** Courses (
  - course\_id INT **PRIMARY** KEY, -- Primary key for Courses
  - course\_name VARCHAR(100) **NOT NULL**
  - );
  -
- -- Create the Students table
- **CREATE TABLE** Students (
  - student\_id INT **PRIMARY** KEY, -- Primary key for Students
  - student\_name VARCHAR(100) **NOT NULL**,
  - course\_id INT, -- Foreign key referencing Courses
  - **FOREIGN** KEY (course\_id) **REFERENCES** Courses(course\_id)
  - );



# RDBMS - Insert data into two tables

-- Insert data into Courses

```
INSERT INTO Courses (course_id, course_name) VALUES  
(1, 'Mathematics'),  
(2, 'Business');
```

-- Insert data into Students

```
INSERT INTO Students (student_id, student_name, course_id) VALUES  
(101, 'Alice', 1),  
(102, 'Bob', 2),  
(103, 'Charlie', 1);
```

# RDBMS - Querying data from 2 tables

```
SELECT Courses.course_name,  
Students.student_name  
FROM Students  
JOIN Courses  
on  
Students.course_id=Courses.course_id;
```

The screenshot shows a database management interface. On the left, a 'SCHEMAS' pane displays a tree view of the database structure. The 'classbd01' database is expanded, showing 'Tables' (Courses, Students), 'Columns', 'Indexes', 'Foreign Keys', 'Triggers', 'Views', 'Stored Procedures', and 'Functions'. The 'Students' table is selected, and its columns (student\_id, student\_name, course\_id) are visible. A tooltip 'Refresh data re-executing the original query' is shown over the 'Columns' folder. The main query editor on the right contains the following SQL query:

```
1 SELECT Courses.course_name, Students.student_name  
2 FROM Students  
3 JOIN Courses  
4 ON Students.course_id=Courses.course_id;
```

Below the query editor, the 'Result Grid' displays the query results. The grid has two columns: 'course\_name' and 'student\_na...'. The results are as follows:

| course_name | student_na... |
|-------------|---------------|
| Mathematics | Alice         |
| Mathematics | Charlie       |
| Business    | Bob           |

# RDBMS – Working with more than 2 tables

```
CREATE TABLE Grades (  
    grade_id INT PRIMARY KEY AUTO_INCREMENT,  
    student_id INT,  
    course_id INT,  
    grade CHAR(2) NOT NULL,  
    FOREIGN KEY (student_id) REFERENCES Students(student_id),  
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
)
```

-- Insert grades for students in their courses

```
INSERT INTO Grades (student_id, course_id, grade) VALUES  
(101, 1, 'A'),  
(102, 2, 'B+'),  
(103, 1, 'B');
```

# RDBMS – Querying more than 2 tables

SELECT

Students.student\_name,  
Courses.course\_name,  
Grades.grade

FROM

Grades

JOIN

Students ON Grades.student\_id = Students.student\_id

JOIN

Courses ON Grades.course\_id = Courses.course\_id;

# RDBMS – Output from more than 2 tables

The screenshot shows a database management interface. On the left, a 'SCHEMAS' pane displays a tree view for 'classbd01', including tables like 'Courses', 'Grades', and 'Students'. The 'Grades' table is expanded, showing columns: 'grade\_id', 'student\_id', 'course\_id', and 'grade'. The main area displays a SQL query:

```
1 • SELECT
2     Students.student_name,
3     Courses.course_name,
4     Grades.grade
5 FROM
6     Grades
7 JOIN
8     Students ON Grades.student_id = Students.student_id
9 JOIN
10    Courses ON Grades.course_id = Courses.course_id;
```

Below the query, a 'Result Grid' shows the output. The grid has columns: 'student\_na...', 'course\_name', and 'grade'. The data rows are:

| student_na... | course_name | grade |
|---------------|-------------|-------|
| Alice         | Mathematics | A     |
| Charlie       | Mathematics | B     |
| Bob           | Business    | B+    |
|               |             |       |
|               |             |       |
|               |             |       |

# Working with the Graph Database

- **Nodes:**

- **Student:**

- Properties: student\_id, student\_name

- **Course:**

- Properties: course\_id, course\_name

- **Grade:**

- Properties: grade

- **Edges:**

- **Enrolled\_in:**

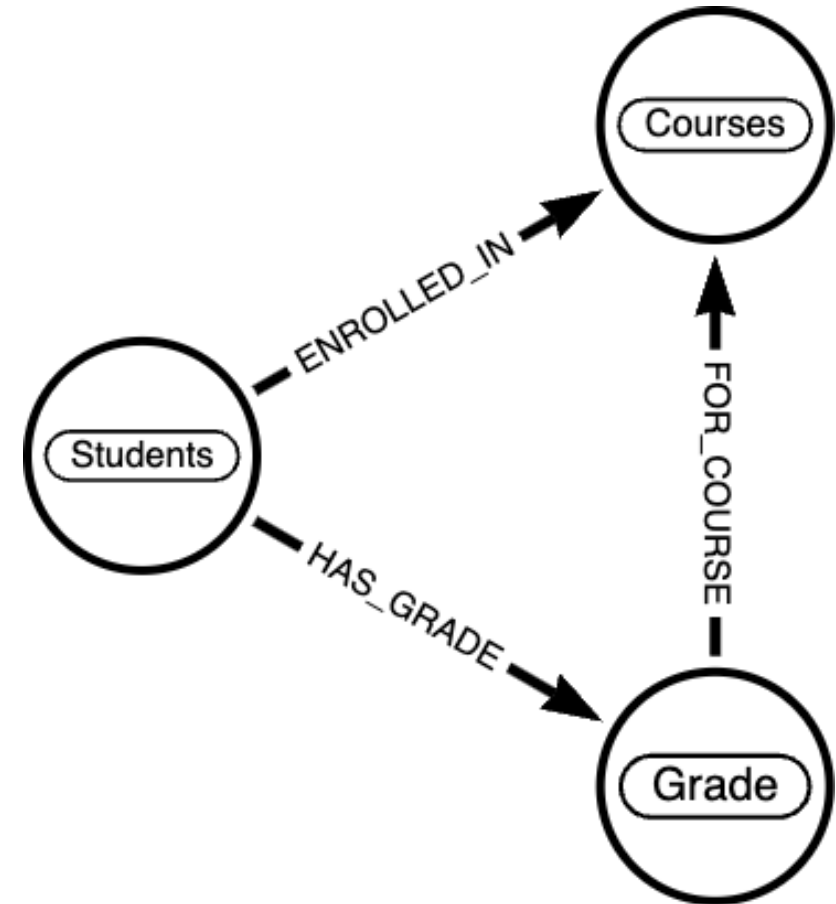
- From Student to Course.

- **Has\_grade:**

- From Student to Grade.

- **For\_course**

- From Grade to Course.





# GraphDB – Cypher Basics

**CREATE** (var:NodeLabel {property1:propertyValue})

**CREATE** (var2:NodeLabel2 {property1:propertyValue})

**CREATE** (var)-[:RELATIONSHIP]-(var2)

**MATCH** (var3:NodeLabel)-[:RELATIONSHIP]-(var4:NodeLabel2)

**RETURN** var3.property1,var4.property1

# GraphDB - Create the nodes and relationships

## CREATE

```
(s1:Student {student_id: 101, student_name: 'Alice'}),  
(s2:Student {student_id: 102, student_name: 'Bob'}),  
(s3:Student {student_id: 103, student_name: 'Charlie'}), //create Students  
(c1:Course {course_id: 1, course_name: 'Mathematics'}),  
(c2:Course {course_id: 2, course_name: 'Business'}), //create courses  
(g1:Grade {grade: 'A'}),  
(g2:Grade {grade: 'B+'}), //create grades  
(g3:Grade {grade: 'B'}), (s1)-[:ENROLLED_IN]->(c1),  
(s2)-[:ENROLLED_IN]->(c2),  
(s3)-[:ENROLLED_IN]->(c1), // student enrolled in class  
(s1)-[:HAS_GRADE]->(g1),  
(g1)-[:FOR_COURSE]->(c1), (s2)-[:HAS_GRADE]->(g2),  
(g2)-[:FOR_COURSE]->(c2), (s3)-[:HAS_GRADE]->(g3),  
(g3)-[:FOR_COURSE]->(c1); // student grades relationship
```

# GraphDB – Querying data

```
MATCH (s:Student)-[:ENROLLED_IN]->(c:Course),  
(s)-[:HAS_GRADE]->(g:Grade)-[:FOR_COURSE]->(c)  
RETURN s.student_name AS Student, c.course_name AS Course, g.grade AS Grade;
```

```
1 \MATCH (s:Student)-[:ENROLLED_IN]->(c:Course),  
2 |      | (s)-[:HAS_GRADE]->(g:Grade)-[:FOR_COURSE]->(c)  
3 RETURN s.student_name AS Student, c.course_name AS Course, g.grade  
   AS Grade;
```



Table



Text



Code

|   | Student   | Course        | Grade |
|---|-----------|---------------|-------|
| 1 | "Alice"   | "Mathematics" | "A"   |
| 2 | "Bob"     | "Business"    | "B+"  |
| 3 | "Charlie" | "Mathematics" | "B"   |

# Exercise – Work with your existing data

- Use the Excel that you worked on to create the table structure
- Create the database on the new AWS account.
- Enter sample data and execute queries